Quantum Annealing for Robust Financial Pattern Discovery

Outperforming conventional classic methods with Quantum L2 Principal Component Analysis in a $25+ Trillion Market

SCQuantathon 2025

Vanderbilt Quantum Initiative

Abstract

We analyze principal component analysis for equity returns by replacing a simple L2 model with an L1 model that can tolerate heavy tails and outliers. We mapped single and multicomponent PCA to Ising models to solve them in multiple ways: a manual simulated annealer, SciPy's dual annealing, and quantum-based approaches. Across a set of assets, L1 yields sharper clustering and more stable principal components. Dual annealing and quantum annealing consistently achieve the lowest objective values, while our manual annealer performs similarly in small spaces but degrades as the number of assets increases. Multi-component optimization produces components that are sensitive to penalties and tolerances, producing inconsistent results. Running QAOA on cloud computers reproduces the classical L1 solutions with greater latency, reflecting the gap in hardware. Overall, L1 PCA via annealing offers a firm alternative to L2 PCA with a clear pathway to applying quantum acceleration as specialized hardware becomes more accessible and easy to implement.

1 Introduction

Principal Component Analysis (PCA) is a cornerstone of finance that allows for the identification of patterns among multi-dimensional sets of data. By calculating and mapping data to a new, smaller set of basis vectors that capture the maximum variation among data, PCA facilitates easier grouping, visualization, and noise reduction.

Traditional PCA relies on an L2-norm (variation based on the square of differences), which is mathematically simple to compute:

$$L_2 = \sum_{i=1}^{n} (y_i - \bar{y})^2$$

However, since the differences are squared, it amplifies the impact of outliers and can skew the component analysis. Therefore, the reality of fat tails in market returns and black swan events typically leads to the misrepresentation of the majority of the data.

In contrast, L1 PCA utilizes the absolute difference between data points, giving a more equally distributed weighting.

$$L_1 = \sum_{i=1}^{n} |y_i - \bar{y}|$$

However, while this reduces the skewing caused by outliers, its properties, including the non-smooth nature of the absolute function, mean that the problem is NP-hard, and thus no known direct solution of polynomial time exists. This means the problem scales quickly to become computationally infeasible through a direct solution.

Annealing provides a method of approximating an optimization through a partially-random walk that iteratively explores nearby solutions from a random start, converging on better solutions while accepting worse solutions with a low probability that decreases over the optimization process, dependent on a temperature function.

The L1 relationships of a dataset can be transformed into an Ising model of binary relations, allowing it to become a binary optimization challenge explorable through annealing.

$$E(b) = b^T J b, \quad b_i \in \{\pm 1\}$$

In this way, it is described as a combination of binary functions and matrices, which is perfectly suited for an annealer that uses binary qubits; particularly, quantum annealing allows for the passage through energy barriers by taking advantage of quantum tunneling to avoid local minima and target the global minimum.

2 Implementation

2.1: Solving L2 with ML/closed form solution

As a baseline, we implemented a simple machine-learning (ML) Python package called sqlearn to solve for the first two principal component vectors of the dataset, when the objective function is defined depending on an L2 rather than L1 model, as this is easier to solve.

Our results using ML converged onto the closed-form solution that is found using the covariance matrix:

$$\Sigma = \frac{1}{n-1} X^T_{centered} X_{centered}$$

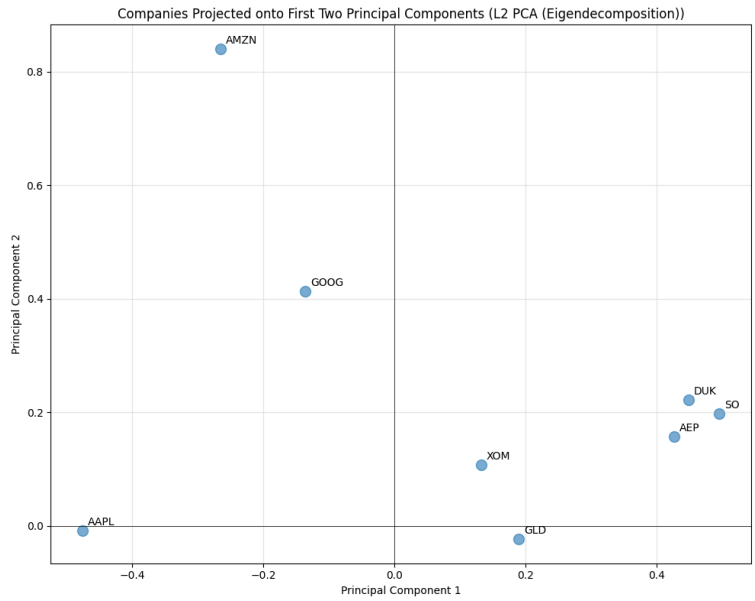
To capture eigenvectors and eigenvalues with the corresponding variance that is explained by that given matrix:

$$\Sigma \omega_i = \lambda_i \omega_i$$

As a projection:

$$Z = X_{centered} W$$

Where Z is the transformed data, projected onto these principal components, giving a new dataset with the principal components as our basis vectors, and all other components in our dataset are ignored.



In this graph, the data has minimum clustering, which implies that this solution of Principal Components might poorly represent the market information. This is particularly notable with the outliers, where they are significantly farther from the rest of the data than in other models, such as a simulated dual annealer (Sec 2.2), which visually represents the concept of outliers skewing the representation of the aggregate of the data compared to PCs. To mathematically verify the results, an L2 PCA model using the numpy library package was employed to compute eigenvectors through eigenvector decomposition. This result yielded the same graph and is shown above.

2.2: Classical L1 PCA

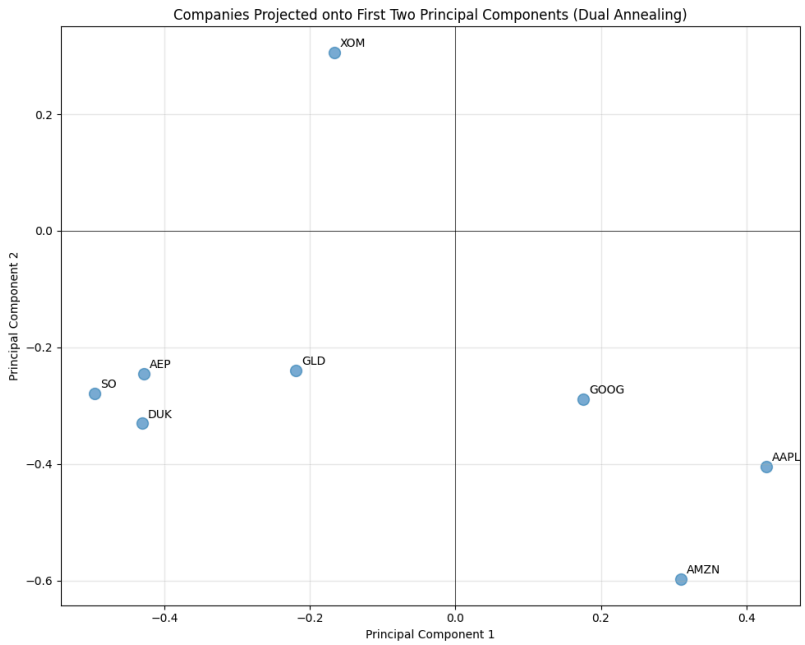
To apply L1 PCA, we first implemented dual annealing using the Python package SciPy, which approximates the optimal principal components using simulated annealing.

Under the hood, SciPy's dual annealing algorithm uses a Cauchy-Lorentz visiting distribution, which generates steps and reduces the artificial 'temperature' as the matrix iterates:

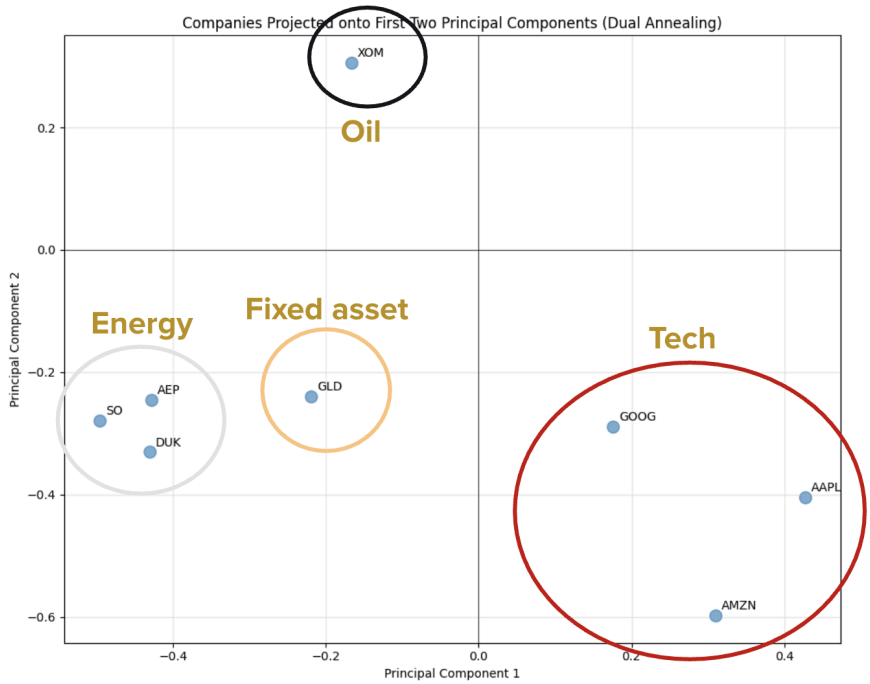
1) Begins with a high temperature that allows for worse moves to be accepted, which allows for exploration of possible solutions
2) Generates candidate moves, which define how far the algorithm jumps from the current point
3) Trial points are compared to the current best. If better, it is automatically accepted. If it is worse, it is tested against a probability that allows it to escape early local minima.
4) The artificial temperature gradually decreases, making the algorithm more selective.
5) Iterate until convergence.

After calculating the first principal component, the influence of this component can be subtracted from the original data, which can then be run again to find the principal component that explains the next greatest variation. This can be iteratively repeated to find k principal components (and is repeated this way for practically all implementations except for the multi-component optimization).

After calculating these new L2 principal components, we projected our data onto them similarly to in section 2.1, resulting in:



This projection results in much clearer clustering, as highlighted below:



While this resulted in clear clustering, which was a positive sign, we found that we had very little control over what happened in this scenario, and it became a sort of "black box." We implemented our own simulated annealing process that could give us the ability to manipulate it more to hypothetically find an optimal solution by better suiting it to the specifics of the exact optimization challenge.

2.3: Solving L1 with manually simulated annealing

Since the SciPy library is based on optimizing continuous values rather than a binary matrix, we hypothesized that a custom-made manual annealing algorithm could be more optimal. This would also give us more control over the specific temperature function and steps being applied, allowing us to optimise it for the particular challenge.

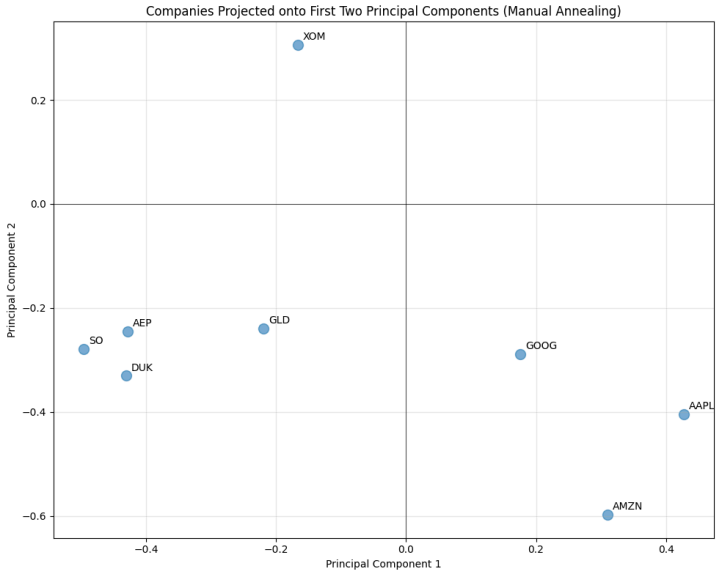
We developed the manual_anneal() function to perform:

1) Initialise random binary matrix B
2) Generate a random nearby matrix B' by flipping a single bit.
3) Compare the energy values between the matrices at the best B so far and B'
4) Accept greater energy values.
5) Accept lower energy values with probability: $e^{(NewValue-OldValue)/T}$

This is then followed by the same iterative process as in section 2.3 to find further principal components.

For the core challenge problem involving eight stocks, this converged upon the same overall energy value of -0.102872 (our data was normalized so that we were actually optimizing to find more

negative solutions). However, this became increasingly less effective as N (number of assets) increased, underperforming compared to the library-based dual annealer, as evidenced later when N was increased.

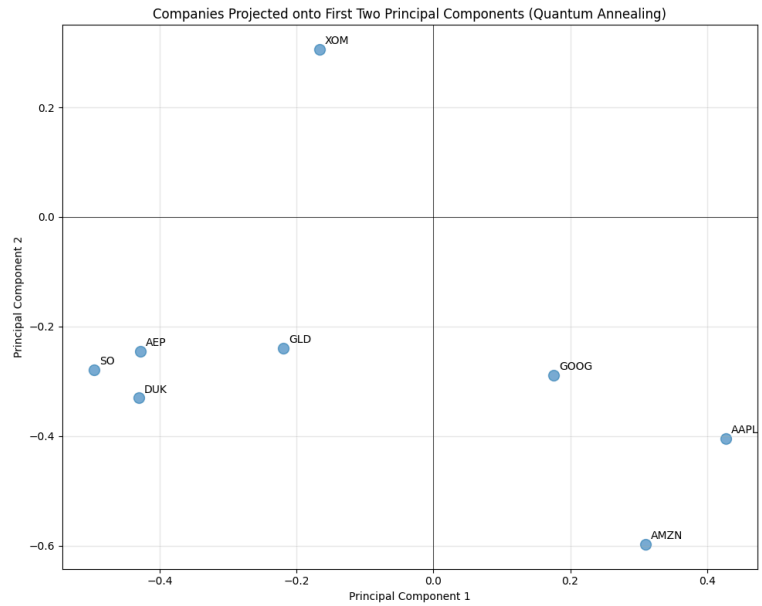


This result is (expectedly) similar to the annealing that relies on dual annealing. While utilizing our own annealing function gave us more control, it made it more difficult to optimize each variable and portion. Since there are multiple different ways to initialize the theoretical "temperature" and different iteration techniques, it was difficult to optimize. In reality, the dual annealing outperformed our manual annealer, though they produced very comparable results.

2.4: Solving L1 by simulating quantum annealing with D-Wave simulation

Quantum annealers are better at escaping some "traps" (local minima) via quantum tunneling through barriers rather than hopping over the energy barriers. We used a D-Wave simulated quantum annealer on classical hardware to try to capture this effect. The algorithm starts with quantum-like fluctuations and high temperature, and (much like a classical annealer), gradually lowers the temperature. While the actual quantum implementation is different, as it involves slowly transforming the states of a system of entangled qubits, it results in a similar optimization, albeit (hypothetically) with this added improvement from quantum entanglement.

While classical hardware does not allow for literal tunneling through energy fields, the algorithm is designed to emulate the impact that tunneling would have: reducing the chance of getting stuck in local minima, making it more likely to descend closer and closer toward a better minimum.



This result is identical to that of the dual annealer. The reality is that using classical hardware will not be able to truly simulate modern quantum effects (as otherwise quantum computers would not be necessary), meaning that a D-Wave simulator outputs similar results to a typical classical annealer. In reality, a quantum computer would have to be utilized to see the full effect of a quantum approach. While the graphs have the same findings, it's suggested that there is potential for growth once quantum annealers are readily available and usable.

2.5: Quantum Multi-component Optimization

Rather than finding components one at a time, we can use multi-component optimization to find binary principal components simultaneously. Each component is a binary $\pm$ 1 over the assets. It uses a similar Ising model construction, but includes a reward for aligning with the J matrix and a penalty for aligning with another matrix that enforces orthogonality. Together, this is represented as:

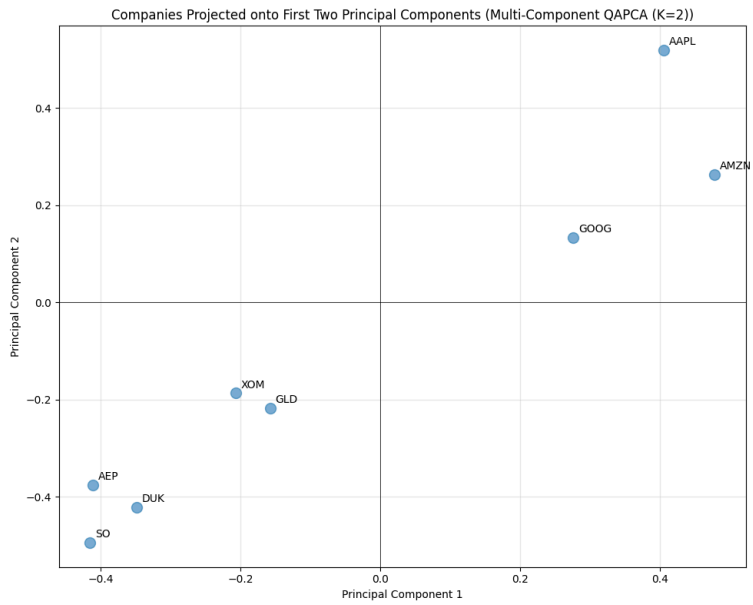
$$J_{ising} = (I_K \otimes KJ) + ((11^{\mathsf{T}} - I_K) \otimes (-\epsilon J))$$

After minimizing this function, the single best sample is output, where each column is a binary component. To transpose binary components into continuous and normalized components, SVD is performed, where:

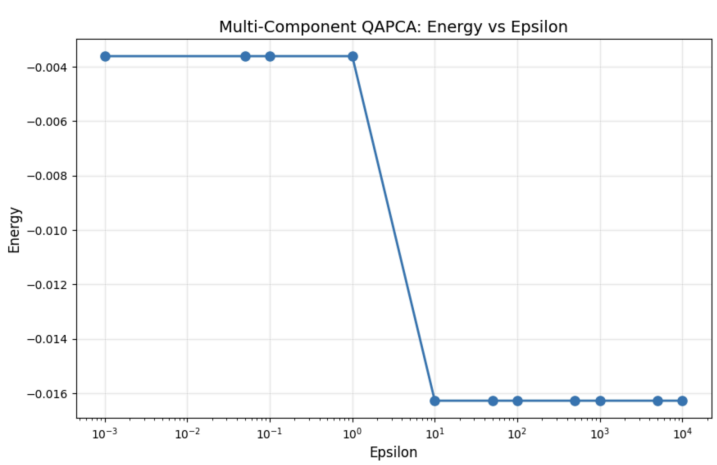
$$c_k = \frac{XR_{:k}}{\|XR_{:k}\|_2}, \quad k = 1, \ldots, K$$

Using this model, components can coordinate, meaning that the algorithm is not order-dependent. For a typical model, differences early on change what is left for later components. By choosing all of the

K values together, PC2 does not depend on the partially random PC1, making joint optimization better to find more accurate components. Also, the solutions in multi-component PCA are orthogonal to one another by nature of a penalty, meaning orthogonality does not have to be processed downstream.



The data in a multi-component quantum annealer was very different than that of other approaches. While some similar clustering emerged (such as with GOOG, AAPL, and AMZN), it is not the same, and could suggest incomplete optimization (although with such a small dataset this hard to differentiate from purely trivial solutions). While we initially hypothesized that this was caused by an incorrect epsilon value (used to determine the strength of the penalty for finding principal components that aren't orthogonal), by altering epsilon systematically, we found that this only resulted in oscillation between two local minima (that are clearly suboptimal compared to previous solvers):



Our final hypothesis was that the possibility space of the multi-component solver is significantly greater than the more confined space of the iterative solver. While this theoretically could reveal more

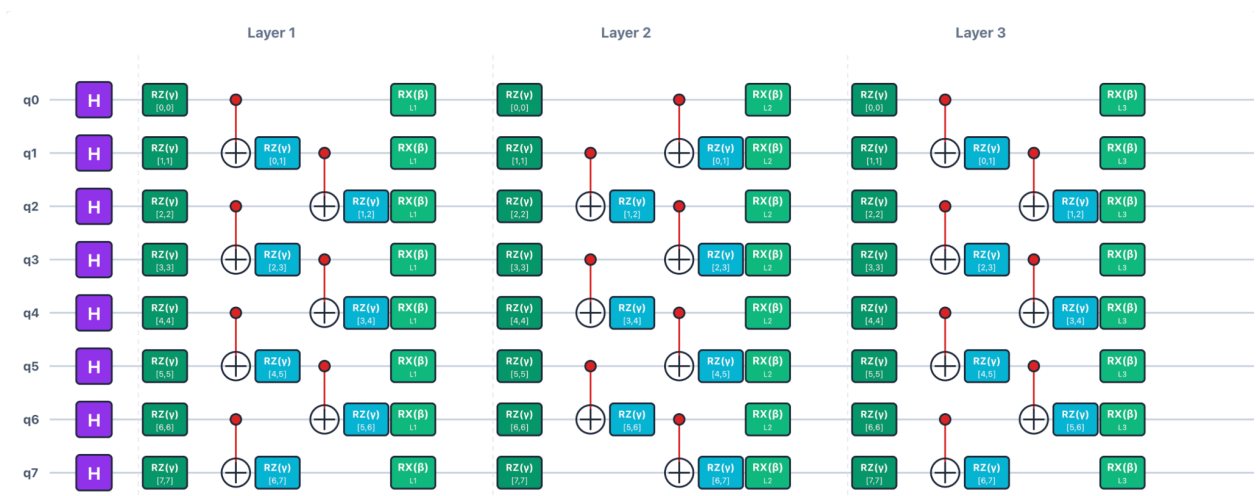
complex relationships relating to more global phenomena, it is computationally inaccurate for the feasible iteration numbers, resulting in convergence on local minima. However, this could also be a result of an implementation error.

2.6: Qiskit

Qiskit is a package centred around using quantum gates to manipulate qubits, which is fundamentally different from the linear-algebra-based classical L1 PCA we have implemented so far. Therefore, to utilize Qiskit, we must use the Quantum Approximate Optimization Algorithm (QAOA) to approximate the annealing algorithm.

Each ticker is represented as a qubit in QAOA annealing. The qubits are passed through a series of gates that manipulate their superposition function.

QAOA Circuit Diagram for 8 Qubits



Gates Used:
H (Hadamard); CNOT(ZZ coupling); RZ(J[i,j] coupling); RZ(J[i,i] diagonal), RX (Mixer), Measure

This circuit describes the implementation of a three-level QAOA algorithm; however, as the pattern is repetitive, we were able to implement the algorithm with the number of layers being an additional variable. In general, three to five layers is the most optimal choice for maximizing good accuracy achieved in a sensible time frame, yielding >90-96% accuracy in a time frame comparable to the classical implementation.

When tested just as a Python, this implementation yielded identical results to the classical L1 PCA, both in terms of the binary matrix and the expectation values, which is expected for a simulation still run on classical hardware (especially since the problem has a small possibility space with only 8 stocks, meaning all optimizers likely converged on a global maximum).

2.7: Qiskit Connection to Circuit

After the Qiskit simulation code was written, the code was modified to be sent and run on IBM's quantum computer. The code was simplified to run a single PCA to focus on the quantum computing aspect of the model. Additionally, this was done to save time in testing this aspect. Setting up the Qiskit program to run on the computer required adding a connection code before executing any of the programs, to establish two key components: the user account and the connection to the computer. The first program involved specifying whether to use the "ibm_cloud" or the "ibm_quantum_platform" instance parameter, paired with a token that would grant access to either of these resources. The first is a faster way to solve problems with quantum needs by hybridizing it with other resources in the IBM Cloud ecosystem. The second is more used to concentrate efforts on running the problems through IBM's actual quantum computing units, or QPUs.

To run the Qiskit program developed in an actual QPU, the second instance was selected, which prompted the API token to be required for this program to run. Once the token was created and pasted into this account setup function, we were already initialized as a user and can now access the QPU by connecting to it via IBM's cloud service. This connection involved the continuous presence of a backend variable that served as a reminder of where the actual code would be processed, instead of being run locally as a simulation. A lot of the logic behind the program is the exact same as the Qiskit program developed, but simply reframed in order to accommodate this cloud-related variable that would consider where the program would be processed.

2.8: Hardware Next Steps

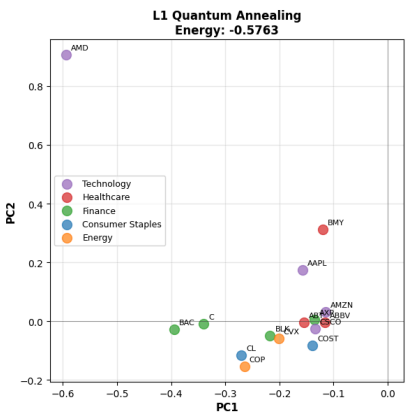
This quantum model converged to the same solution as the previous models (the likely global optimum). This proves that the real quantum approach is at least as good as the other methods. While this doesn't prove superiority, it provides an initial architecture to be extended. Hypothetically, implementing this in a real quantum annealer (such as those developed by D-Wave) would result in the emergence of further improvements from quantum tunnelling that are not evident in the classical simulations, or even these smaller quantum simulations. This hardware implementation is the clear next step to actually determine the feasibility of quantum superiority, which was not possible within the restrictions of the quantathon.

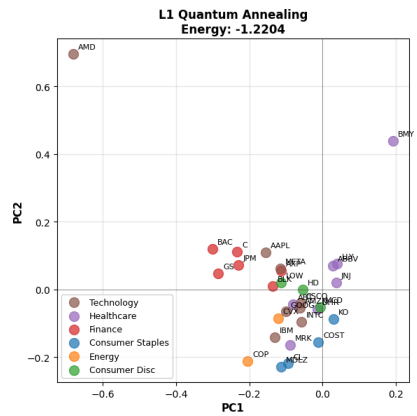
3 Extension

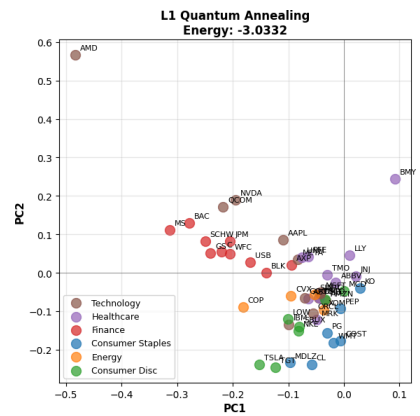
3.1: Increasing Assets

As we iterated, we increased the number of stocks, which allowed us to visualize a greater market sector, and also take into consideration the influence of outliers, and a larger data set (also demanding more processing power).

Consider the L1 Quantum Annealing approach at different numbers of assets (15, 30, 50):



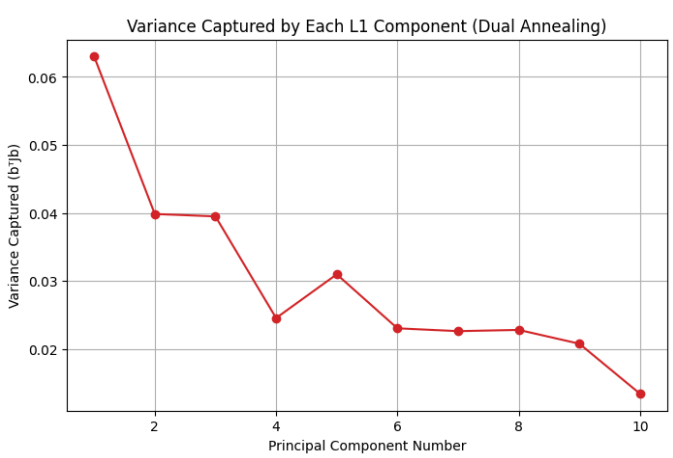




As we include more assets, the first two L1 principal components remained fairly stable and continued to show sector structure where assets became clustered based on their sector. The L1 variance explained by PC1 and PC2 was high, indicating that adding more assets added depth, but did not fundamentally change which factors were most significant.

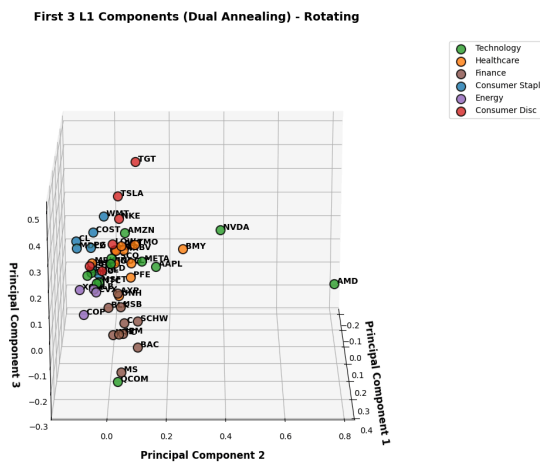
3.2: More Principal Components

Modeling and visualizing two principal components at a time is simple and chosen for its ease of visualization, but inherently does not capture all of the data. As such, we increased the number of principal components that we tested for, measuring the impact of each one on the variance:



Here, it is shown that the first three principal components explain a large portion of the variance in the data. Since there is a drop-off after the third principal component, the fourth, etc., would not necessarily add much value; additionally, adding more components adds the potential for more noise to be added into the system, which would clutter our results. Therefore, three principal components are ideal for modeling all of our data and explaining the largest amount of variance while reducing unnecessary noise.

Consider the L1 Quantum Annealing Case:

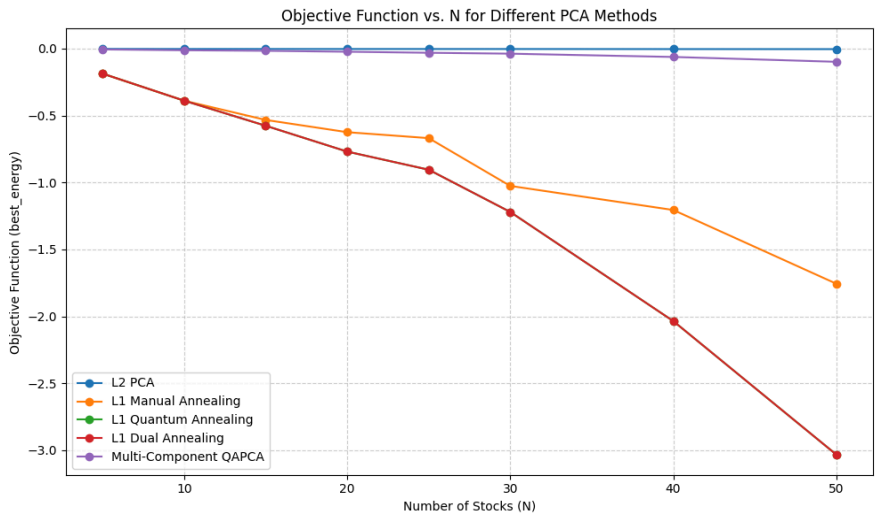


Except for some outliers, the data is much better clustered (highlighting clearly outliers, and clear clusters such as finance), which shows that the data in this case is better represented when presented against three principal components, when compared to two. Colouring is based on an additional manual sector labelling.

Hypothetically, it is possible to include a fourth dimension (time) where we track the three-dimensional components in response to the passage of time; however, there was minimal variance in the given timespan, and this would be not transfer well to a video-based plot with time as a dimension since we only have discrete data points rather than continuous data.

4 Conclusion

4.1: Comparison of Methods



** where L1 Quantum Annealing lies on the same line as L1 Dual Annealing

In this case, L1 Dual Annealing and L1 Quantum provide better results than other approaches since the goal of the function is to minimize the Objective Function. As more assets are added, the gap widens, which is logical because these annealing approaches can better handle more assets, particularly in datasets that include outliers.

Notably, L1 Quantum Annealing yields the same results as L1 Dual Annealing; in this case, it is logical because the quantum annealing is simply a simulation of how a true quantum annealer would work. A classical simulation is directly comparable to a well-functioning classical solution, which is one of the key limitations on our ability to implement our quantum solution.

Based on our findings, the ideal classical approach currently rivals a simulated quantum approach. However, we were constrained by hardware, where our quantum annealer currently has to be run on a classical simulator. In theory, a quantum approach on a specialized quantum computer (like a D-Wave computer) has the potential to outperform a classical approach.